Feature Engineering and Model Evaluation for E-commerce Dataset

JOU-YI LEE
UC Santa Cruz, M.S. in NLP
65 Rio Robles E
San Jose, CA 95134
jlee1039@ucsc.edu

Abstract

Using an e-commerce dataset, this research provides an in-depth investigation of feature engineering and model evaluation for a multi-class classification problem. Three distinct feature engineering approaches were used to assess three classifiers: Support Vector Machine (SVM), Logistic Regression, and Decision Tree. Accuracy, training and inference times, and the macro-average F1 score were all the performance parameters examined. To find the best model configurations and evaluate the durability of the model across several classes, hyperparameter tweaking was also carried out using Optuna and OneVsRest assessment.

1 Introduction

In this assignment, we aim to solve a 4-class classification (Household, Books, Electronics, Clothing& Accessories) problem using an e-commerce dataset that includes item categories and descriptions. Class Distribution like the Figure 1. We employed three classifiers (SVM, Logistic Regression, Decision Tree) and evaluated them with three different feature engineering techniques: Bag-of-Words (BoW), TF-IDF, and N-gram. The dataset was split into 70% training, 10% validation, and 20% testing, ensuring reproducibility by using a fixed random seed.

The training, validation, and test set sizes are as follows:

- Training set size: 35296
- Validation set size: 5042
- Test set size: 10086

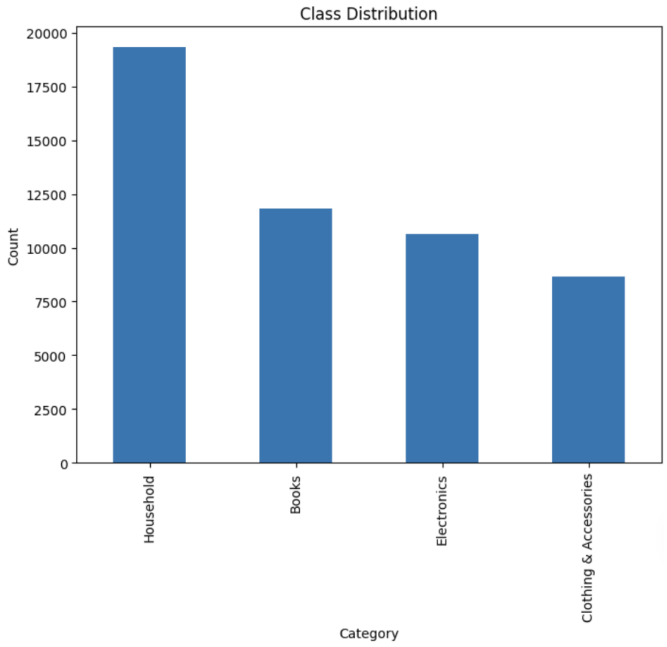

Figure 1: Class Distribution

2 Feature Engineering Techniques

Three feature engineering methods were applied to the text data:

- ***CountVectorizer* for Bag-of-Words**: from scikit-learn. By transforming the text input into a matrix of token counts, this method enables the model to record information on word frequency. In the *review/text* column, we eliminated rows with missing data then used the *CountVectorizer* to choose the 3,000 most common features.
- **TF-IDF using TfidfVectorizer**: The second feature employed the *TfidfVectorizer*,which determines the Term Frequency-Inverse Document Frequency (TF-IDF) values for every word in the review text. This is done in natural language processing and information retrieval tasks to evaluate a word's significance inside a document and its

prevalence throughout the corpus. In *TfidfVectorizer*, Higher priority is given to uncommon words in the entire corpus that often occur in a particular document. We chose 3,000 max features that were kept for this project in this part.

- **N-gram**: A feature that considers the sequence of words to capture contextual information. Using `CountVectorizer` with `ngram_range=(1, 2)`, it generates a word frequency vector containing both 1-gram and 2-gram features. Specifically:

  1-gram: 1-gram represents the frequency of individual words. For example, in this description, words like "Paper", "Design", "Framed", "Wall" will be counted separately.

  2-gram: 2-gram represents the combination of two adjacent words. For instance, "Paper Plane", "Plane Design", "Design Framed", "Framed Wall" are combinations of two adjacent words that will also be counted.

  `CountVectorizer` will generate a bag-of-words model that contains these 1-gram and 2-gram terms along with their respective frequencies. For example, if "Paper Plane" appears once and "Design Framed" appears once, these 2-grams will be recorded and become part of the feature vector.

  Example of Generated N-grams:

  Original Text:

  > Paper Plane Design Framed Wall Hanging Motivational Office Decor Art Prints (8.7 X 8.7 inch) - Set of 4 Painting made up in synthetic frame with uv textured print which gives multi effects and attracts towards it. This is a special series of paintings which makes your wall very beautiful and gives a royal touch. .... It would be treasured forever by whoever your lucky recipient is. Liven up your place with these intriguing paintings that are high definition hd graphic digital prints for home, office, or any room.

  Based on this text content, here are examples of 1-gram and 2-gram features:

1-gram (Single Word Features)	**2-gram (Word Pair Features)**
"Paper"	"Paper Plane"
"Plane"	"Plane Design"
"Design"	"Design Framed"
"Framed"	"Framed Wall"
"Wall"	"Wall Hanging"
"Hanging"	"Motivational Office"
"Motivational"	"Office Decor"
"Office"	"Decor Art"
"Decor"	"Art Prints"
"Art"	"Printed Painting"
"Prints"	"high definition"
"Painting"	"unique painting"
"technology"	"true colours"
"colors"	"to hang"
"beautiful"	"special series"
"unique"	"to your"
"recipient"	"etc."
"etc."	

3 Results and Analysis

3.1 Performance Comparison

The accuracy, macro-average F1 scores, training time, and inference time for each model with different features are shown below.

Model	Accuracy	Macro F1	Training(s)	Inference(s)
LR (BoW)	0.9740	0.9742	17.22	0.0125
LR (TF-IDF)	0.9676	0.9678	17.55	0.0110
LR (N-gram)	0.9750	0.9752	138.17	0.0571
DT(BoW)	0.9545	0.9553	34.85	0.0163
DT(TF-IDF)	0.9477	0.9482	42.84	0.0423
DT(N-gram)	0.9535	0.9544	152.72	0.0699
SVM(BoW)	0.9734	0.9735	13.63	0.0145
SVM(TF-IDF)	0.9783	0.9786	2.22	0.0145
SVM(N-gram)	0.9751	0.9750	36.68	0.0434

Table 1: Performance Comparison of Different Models and Feature Engineering Techniques

Confusion Matrix for Multi-Class Classification

In multi-class classification, the confusion matrix expands into an $N \times N$ matrix, where N is the number of classes. Each row of the matrix represents the **actual class**, while each column represents the **predicted class** by the model.

- **Diagonal elements**: Show the number of correct classifications by the model for each class. A higher value indicates higher accuracy for that class.

- **Off-diagonal elements**: Represent misclassifications, where samples of one class are incorrectly predicted as another class.

The general structure of a multi-class confusion matrix is as follows:

$$\begin{bmatrix} \text{TP}_{11} & \text{FP}_{12} & \text{FP}_{13} & \cdots & \text{FP}_{1N} \\ \text{FN}_{21} & \text{TP}_{22} & \text{FP}_{23} & \cdots & \text{FP}_{2N} \\ \text{FN}_{31} & \text{FN}_{32} & \text{TP}_{33} & \cdots & \text{FP}_{3N} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \text{FN}_{N1} & \text{FN}_{N2} & \text{FN}_{N3} & \cdots & \text{TP}_{NN} \end{bmatrix}$$

Each element (i, j) in the matrix has the following meaning:

- (i, i): Diagonal elements represent the model's **True Positives** for class i, meaning the samples from class i that are correctly predicted as class i.
- (i, j) when $i \neq j$: Off-diagonal elements represent misclassifications, where samples from the actual class i are incorrectly predicted as class j, referred to as **False Positives** or **False Negatives** depending on the perspective.

For each model, the following confusion matrices illustrate the prediction performance.

$$\text{Logistic Regression(BoW)}: \begin{bmatrix} 2305 & 10 & 12 & 37 \\ 9 & 1698 & 8 & 19 \\ 28 & 3 & 2036 & 58 \\ 36 & 14 & 28 & 3785 \end{bmatrix}$$

In this matrix:

- **Row 1** (actual Class 1):
  - (1,1) = 2305: The model correctly predicted 2305 samples as Class 1.
  - (1,2) = 10: The model misclassified 10 samples of Class 1 as Class 2.
  - (1,3) = 12: The model misclassified 12 samples of Class 1 as Class 3.
  - (1,4) = 37: The model misclassified 37 samples of Class 1 as Class 4.
- **Row 2** (actual Class 2):
  - (2,1) = 9: The model misclassified 9 samples of Class 2 as Class 1.
  - (2,2) = 1698: The model correctly predicted 1698 samples as Class 2.
  - (2,3) = 8: The model misclassified 8 samples of Class 2 as Class 3.
  - (2,4) = 19: The model misclassified 19 samples of Class 2 as Class 4.
- **Row 3** (actual Class 3):
  - (3,1) = 28: The model misclassified 28 samples of Class 3 as Class 1.
  - (3,2) = 3: The model misclassified 3 samples of Class 3 as Class 2.
  - (3,3) = 2036: The model correctly predicted 2036 samples as Class 3.
  - (3,4) = 58: The model misclassified 58 samples of Class 3 as Class 4.
- **Row 4** (actual Class 4):
  - (4,1) = 36: The model misclassified 36 samples of Class 4 as Class 1.
  - (4,2) = 14: The model misclassified 14 samples of Class 4 as Class 2.
  - (4,3) = 28: The model misclassified 28 samples of Class 4 as Class 3.
  - (4,4) = 3785: The model correctly predicted 3785 samples as Class 4.

$$\text{Logistic Regression (TF-IDF)}: \begin{bmatrix} 2265 & 16 & 15 & 68 \\ 7 & 1695 & 7 & 25 \\ 35 & 3 & 2006 & 81 \\ 23 & 18 & 29 & 3793 \end{bmatrix}$$

$$\text{Logistic Regression (N-gram)}: \begin{bmatrix} 2306 & 9 & 9 & 40 \\ 11 & 1698 & 6 & 19 \\ 29 & 3 & 2042 & 51 \\ 34 & 15 & 26 & 3788 \end{bmatrix}$$

Discussion:

The SVM with TF-IDF features outperformed the other models in terms of accuracy (0.9783) and macro-average F1 score (0.9786), indicating that the TF-IDF features successfully identified significant word differences between classes. N-gram features, despite their higher computing cost, may be able to capture more context. With an accuracy of 0.9750 and a macro F1 score of 0.9752, the Logistic Regression (N-gram) model also fared well, requiring much more training time (138.1704 seconds) than the other models.

With a maximum accuracy of 0.9545 (BoW), the Decision Tree models, on the other hand, performed worse

across all feature categories. This might be because decision trees are inherently limited in handling high-dimensional sparse data. Regarding accuracy and F1 score, the SVM model with TF-IDF fared better overall than the other models. This is probably because the TF-IDF feature highlights uncommon but significant phrases, which makes it a good fit for linear classifiers like SVM.

If efficiency is needed, the Logistic Regression model with BoW is a potential substitute because it also showed outstanding performance with fewer computational requirements.

Optuna Hyperparameter Optimization (Commented Out)

Although the hyperparameters were predefined, Optuna was originally used to optimize these hyperparameters. Optuna systematically explored a range of hyperparameters and iteratively tested different combinations to maximize the validation accuracy, ultimately identifying the best parameter values.

The best-performing models were further optimized using hyperparameter tuning:

- **Logistic Regression**: $C = 53.998$, Solver: lbfgs
- **Decision Tree**: max_depth = 32, min_samples_split = 6
- **LinearSVC**: $C = 6.833$, max_iter = 7783

Model	Acc.	Macro F1	Train Time	Infer Time
Logistic (BoW)	0.9727	0.9730	124.01	0.0151
Logistic (TF-IDF)	0.9804	0.9807	59.83	0.0120
Logistic (N-gram)	0.9753	0.9754	555.25	0.0610

Table 2: Performance of Logistic Regression Models

Model Results Summary and Results Explanation

Logistic Regression

- **Best Performance**: In Logistic Regression Using TF-IDF features achieved the best accuracy of 0.9804 and a macro F1 score of 0.9807,also low training and inference times.
- **High Training Time for N-gram**: The model with N-gram features have the longest training time (555 seconds) but achieved high accuracy and F1 scores.

Decision Tree

- **Consistent Performance Across Features**: All three feature sets had similar performance, with accuracy ranging from 0.8919 to 0.8924.
- **Fastest Training and longest training**: Using BoW features had the shortest training time (13.8 seconds),also N-gram features had longer training time (37.1 seconds).

LinearSVC

- **Best Model with TF-IDF**: The TF-IDF model had the shortest training time (5.5 seconds) and achieved the best accuracy of 0.9799 and macro F1 score of 0.9802.
- **Increased Training Time with N-gram**: The N-gram model increased training time (218.7 seconds) but achieved lower accuracy than the TF-IDF model.

Overall Analysis Best Model

Logistic Regression was selected as the best-performing model due to its relatively short execution time and its ability to provide stable and accurate results, especially when data is linearly separable. In this run, we achieved an accuracy of **0.9805**, meaning that 98.05% of the predictions on the test samples were correct.

- **C**: 95.79830565462413
  C is a regularization parameter in the Logistic Regression model, controlling the strength of the penalty. A larger C value reduces the regularization effect, which can indicate better linear separability in the data or a lower risk of overfitting.
- **Solver**: liblinear
  The `solver` parameter defines the optimization algorithm used for Logistic Regression. Here, `liblinear`, a coordinate descent-based optimization algorithm, was selected. This solver is well-suited for small to medium-sized datasets and works particularly well for binary and OneVsRest multi-class classification tasks.

  `liblinear` is often effective when dealing with TF-IDF features, as it supports the regularization needs of Logistic Regression and can handle sparse data efficiently.

Reasons for Logistic Regression's Superior Performance

In our model tuning and comparison, Logistic Regression emerged as the top performer, reaching an accuracy of 98.05%. The potential reasons for this include:

- **Linear Separability**: The data likely has good linear separability, enabling Logistic Regression to accurately distinguish between different classes.
- **Regularization Effect**: The regularization parameter C was optimally tuned, providing a model complexity suitable for the data, which prevents both overfitting and underfitting.
- **Support for Sparsity**: Since TF-IDF features were used, the data may be sparse. The `liblinear` solver is particularly effective in handling such sparse data, contributing to the model's high performance.
- **Optimal Model and Feature Combination**: In terms of accuracy and macro F1 score, **Logistic Regression (TF-IDF)** is the best choices. But and **LinearSVC (TF-IDF)** also have really good performance. Also Both have high accuracy and low training and inference times.
- **Recommended Model for Quick Training**: For fast training, the best option is **LinearSVC (TF-IDF)** , with the shortest training time and excellent performance. If longer training time is acceptable, **Logistic Regression (TF-IDF)** provides similarly high accuracy and stable performance.

4 OneVsRest Evaluation

Generally speaking, classification algorithms come in two varieties: (1) Binary classification algorithms. (2) Multi-class classification algorithms.

However, For binary classification challenges, many well-known classification techniques were created specifically. Therefore, multi-class classification issues cannot be solved directly using these widely used classification algorithms. Multi-class classification problems can be divided into numerous binary classification problems using heuristic techniques. Multi-class classification algorithms. OneVsRest is a classification strategy used for multi-class problems, also known as "One-vs-All." Assuming there are four classes (Class 1, Class 2, Class 3, Class 4), OneVsRest will train a separate binary classifier for each class. The specific steps are as follows:

For each class i, a binary classifier is created, marking all samples of that class as "positive" and all other classes' samples as "negative."

- **Class 1**: Accuracy = 0.9781, Macro F1 Score = 0.9689. This shows an accuracy close to 98%. The F1 score indicates a balanced performance in terms of precision and recall.
- **Class 2**: Accuracy = 0.9869, Macro F1 Score = 0.9768. The highest accuracy, close to 99%, indicating that the model is highly accurate in identifying this class.
- **Class 3**: Accuracy = 0.9759, Macro F1 Score = 0.9628. The F1 score is slightly lower than for other classes, suggesting that either the precision or recall for Class 3 may be slightly lower.
- **Class 4**: Accuracy = 0.9739, Macro F1 Score = 0.9724. Both accuracy and F1 score are relatively balanced for this class.

Average Accuracy (OneVsRest): 0.9787 Average Macro F1 Score (OneVsRest): 0.9702 it demonstrating the model's stability and reliability.

4.1 ROC Curve

The ROC curves for each class are close to the top-left corner, and the AUC (Area Under the Curve) values are near 1.0. This shows that the model has a high true positive rate and low false positive rate when distinguishing each class.

4.2 Precision-Recall Curve

In each class's Precision-Recall curve, both precision and recall remain high and approach 1.0, indicating that the model maintains high accuracy when identifying samples for each class, with few errors.

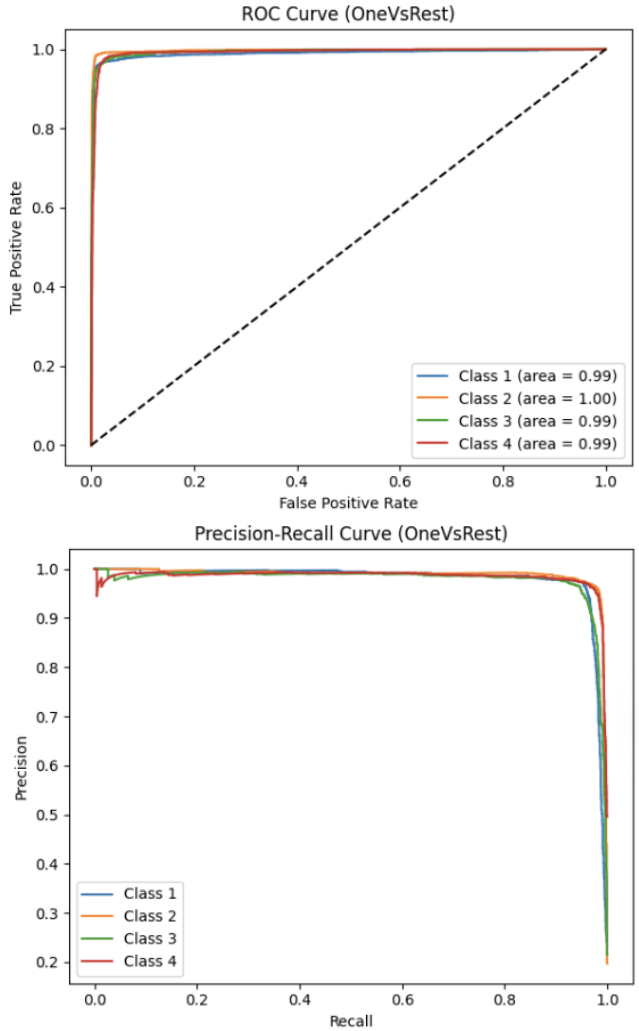

Figure 2: ROC and Precision-Recall Curves (OneVsRest)

5 Conclusion

This study focuses on solving multi-class classification problems using e-commerce datasets with item categories and descriptions. Three classifiers—logistic regression, support vector machine (SVM), and decision trees—were evaluated using three feature engineering techniques: bag-of-words (BoW), TF-IDF, and N-grams. In this project, we optimized Optuna's hyperparameter optimization and evaluated using the OneVsRest strategy to determine the optimal model configuration. Confusion matrices evaluate model performance, demonstrating each model's ability to handle multiple classes accurately.

The OneVsRest method effectively evaluates the precision and recall of each class, and the ROC and Precision-Recall curves show that the TP rate is high and the FP rate is low between classes. Based on OneVs-Rest evaluation and support from performance metrics, optimal model configurations and feature engineering methods are selected to maximize classification accuracy and reliability.

The logistic regression model with TF-IDF features performed best, with an accuracy of 98.05% and an average macro F1 score of 0.9807. This model's success can be attributed to the data's linear separability and the TF-IDF feature's ability to highlight unique terms in text, making it well-suited for linear classifiers such as Logistic Regression. The SVM model with TF-IDF also performs well, especially in terms of efficiency and maintaining high accuracy, making it a competitive choice when training time is a key consideration. Logistic regression (TF-IDF) was the best model for accuracy, stability, and efficiency in handling sparse data. At the same time, SVM with TF-IDF remains a viable option when efficiency is a priority.

References

- GeeksforGeeks. *One-vs-Rest strategy for Multi-Class Classification*. Retrieved from

  ://www.geeksforgeeks.org/one-vs-rest-strategy-for-multi-class-classification/

- Optuna. *Optuna - A hyperparameter optimization framework*. Retrieved from https://optuna.org/